

ПОЛИМОРФИЗЪМ - Наследяване на конструктори – задача Сгради

Конструкторът е специален метод, който носи името на класа, в който се намира, но не връща тип. Може да имаме няколко конструктора, които да приемат различни входни данни (input variables), както разгледахме в задачата за Area, а компилаторът ще прецени кой точно да се изпълни според типа, последователността и броя на тези подадени input variables. Специален случай е празният конструктор, който е по подразбиране от intellisense и няма входни данни – в нашия случай public Building() {}. Между скобите не подаваме никакви променливи, не ги свързваме, но може да извършим действие – да отпечатаме съобщение с sw.

Нека създадем родителски клас Building – задачата ще бъде за сгради – къщи и апартаменти.

```
public class Building
{
    5 references
    public string Type { get; set; }
    6 references
    public string Color { get; set; }
    4 references
    public int Year { get; set; }
    конструктори
    2 references
    1) public Building() => Console.WriteLine("Празен конструктор Building()"); // => съкратен запис вместо {}
    1 reference
    2) public Building(string color)
    {
        Type = "неизвестен";
        Color = color;
        Console.WriteLine($"Типът на сградата е {Type}, цветът е {Color}, годината не е въведена.");
    }
    2 references
    3) public Building(string type, string color, int year)
    {
        Type = type;
        Color = color;
        Year = year;
        Console.WriteLine($"Сградата е {Type}, цвят {Color}, година {Year} г.");
    }
}
```

(input variables)
(входни променливи)

В родителския клас имаме 3 конструктора: 1 празен (), без никакви входни променливи, който само отпечата съобщение, втори, който приема един параметър и трети, който приема най-много. Ако създадем в Main() нов обект, на който не подадем никакви входни данни между скобите ()

```
Building b1 = new Building();
```

той ще бъде обработен от празния конструктор и ще изведе съобщението на екрана.

Може да създадем още един обект, за който да подадем цвят – той ще бъде разпознат като правилен от втория конструктор `Building b2 = new Building("бежов");` и евентуално трети обект `Building b3 = new Building("читалище", "бежов", 1987);` който, разбира се, ще бъде обработен от третия конструктор.

Това е полиморфизъм (гръцки) от поли = много + морфос = форма, т.е. много форми.

Важно е да отбележим, че без празния конструктор, няма да можем да създадем празен обект.

След това, нека създадем 2 наследника: House и Apartment. Нека всеки от тях има по 1 специфично свойство и по 2 конструктора – един празен и един, който изисква голям брой различни входни данни.

Запомнете: **празният конструктор ни позволява да създаваме нови празни обекти!**

Празният конструктор в наследника е единственият, който няма нужда от изречно унаследяване на празния конструктор на родителя чрез : base(). Можем да го изпишем, можем и да го пропуснем, защото е празен. Но само той!!! Принципно, **конструктори НЕ се унаследяват в C#.** Те се посочват изречно чрез : base(...) И отново – няма нищо смущаващо в случая със знака =>, той просто спестява чифт къдрави скоби {}. Това, че конструкторът е празен, т.е. няма никакви входни данни (input variables), не означава, че в тялото му не могат да се извършват действия като sw и други!

```
public class House : Building
{
    2 references
    public int Floors { get; set; }

    1 reference    съкратен запис без : base()
    public House() => Console.WriteLine("Празен конструктор House()"); // : base() в случая е излишно
    1 reference
    public House(string type, string color, int year, int floors) : base(type, color, year)
    {
        Floors = floors;
        Console.WriteLine($"Къщата е с пълни характеристики: цвят {Color}, година {Year} г., етаж {Floors}");
    }
}

6 references
public class Apartment : Building
{
    2 references
    public bool Terraces { get; set; }

    1 reference    еквивалентен запис
    public Apartment() : base() { Console.WriteLine("Празен конструктор Apartment()"); }
    1 reference
    public Apartment(string type, string color, int year, bool terraces) : base(type, color, year)
    {
        Terraces = terraces;
        Console.WriteLine($"Пълни данни за {Type}: {Color}, {Year} г., " + (Terraces ? "има " : "няма ") + "тераси.");
    }
}
```

И така, нека видим, ако създадем различни обекти в Main() какво ще се случи като последователност - кои конструктори ще бъдат извиквани и в каква последователност?

Знаем, че при всяко създаване на нов обект с **new**, се извиква съответния конструктор (който свързва public properties = input variables), затова именно в конструктора е най-удачното място да поставим static брояч Counter++ и TotalSum+=Sum, които не принадлежат на 1 конкретен обект, а на целия клас.

И така:

```
static void Main(string[] args)
{
    Building b1 = new Building();           // РОДИТЕЛ
    Building b2 = new Building("бежов");
    Building b3 = new Building("читалище", "бежов", 1987);
    Console.WriteLine(new string('~', 33));

    House h1 = new House();                 // НАСЛЕДНИК 1
    House h2 = new House("къща", "охрана", 1998, 3);
    Console.WriteLine(new string('~', 33));

    Apartment a1 = new Apartment();         // НАСЛЕДНИК 2
    Apartment a2 = new Apartment("апартамент", "зелен", 2004, true);
}
```

Резултат:

```

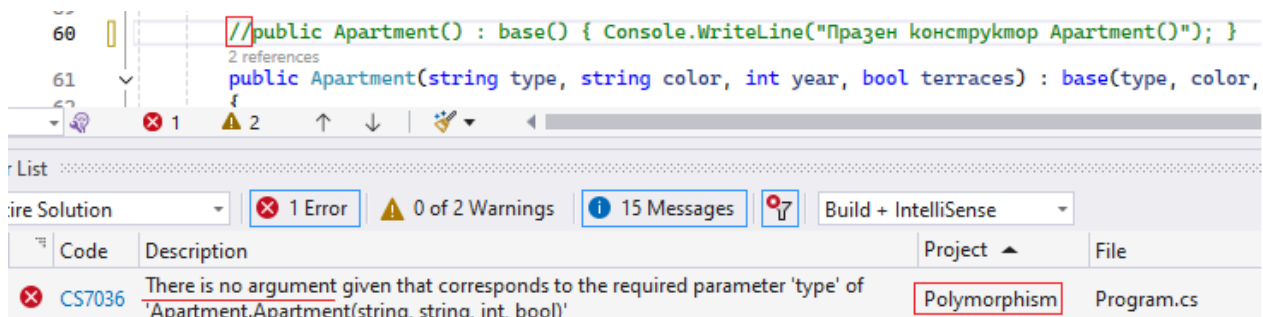
C# Microsoft Visual Studio Debug Console
Празен конструктор Building() РОДИТЕЛ
Типът на сградата е неизвестен, цветът е бежов, годината не е въведена.
Сградата е читалище, цвят бежов, година 1987 г.
~~~~~
Празен конструктор Building() РОДИТЕЛ
Празен конструктор House() наследник 1
Сградата е къща, цвят охра, година 1998 г.
Къщата е с пълни характеристики: цвят охра, година 1998 г., етажи 3
~~~~~
Празен конструктор Building() РОДИТЕЛ
Празен конструктор Apartment() наследник 2
Сградата е апартамент, цвят зелен, година 2004 г.
Пълни данни за апартамент: зелен, 2004 г., има тераси.
D:\Exercises\11 class\Inheritance\Polymorphism\bin\Debug\net8.0\Polymorph
Press any key to close this window . . .

```

Като извод: празният конструктор на наследника първо се качва при родителя, изпълнява неговото съобщение, а след това своето.

Това е смисълът на `base` – разширява функционалност, като взема от родителя и допълва.

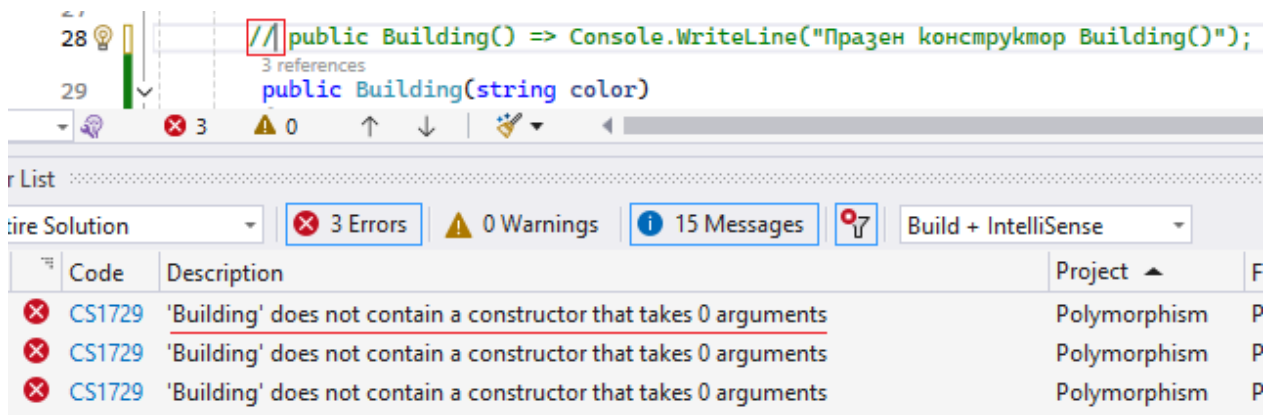
Нека сега тестваме, ако закоментираме празния конструктор в някой от наследниците, каква би била изписаната грешка:



Полиморфизъм на Line 18

```
18 Apartment a1 = new Apartment(); // НАСЛЕДНИК 2
19 Apartment a2 = new Apartment("апартамент", "зелен", 2004, true);
```

Нека сега закоментираме и празния конструктор в родителя – последствията са още по-сериозни:



3 грешки с още по-ясен код, че класът Building (родителят) не съдържа конструктор с 0 аргумента (празен).

В Main() се подчертава обектът, който не може да бъде създаден празен:

```
Building b1 = new Building(); // РОДИТЕЛ
```

Както и двете места, в които наследниците имат празни конструктори, които не разпознават родителския. Можем да прочетем в кои редове Line са грешките или се ориентираме по скрулера.

```
public class House : Building
{
    2 references
    public int Floors { get; set; }

    1 reference
    public House() => Console.WriteLine("Празен конструктор House()"); // : base() в случая е излишно
    1 reference
    public House(string type, string color, int year, int floors) : base(type, color, year)
    {
        Floors = floors;
        Console.WriteLine($"Къщата е с пълни характеристики: цвят {color}, година {Year} г., етажи {Floors}");
    }
}

6 references
public class Apartment : Building
{
    2 references
    public bool Terraces { get; set; }

    1 reference
    public Apartment() : base() { Console.WriteLine("Празен конструктор Apartment()"); }
    1 reference
    public Apartment(string type, string color, int year, bool terraces) : base(type, color, year)
    {
        Terraces = terraces;
        Console.WriteLine($"Пълни данни за {Type}: {Color}, {Year} г., " + (Terraces ? "има " : "няма ") + "тераси.");
    }
}
```

